Composition & Inheritance

Viska Mutiawani, M.IT

Reuse Class

- Ada 2 cara untuk reuse (menggunakan kembali) class yang telah dibuat:
  - Composition (has-a)
  - Inheritance (is-a)

Composition vs Inheritance

- Composition : aggregation
- Composition : class baru disusun dari class yang telah ada.
- Inheritance : pewarisan
- Inheritance : class baru dibuat berdasarkan modifikasi dari class yang telah ada.

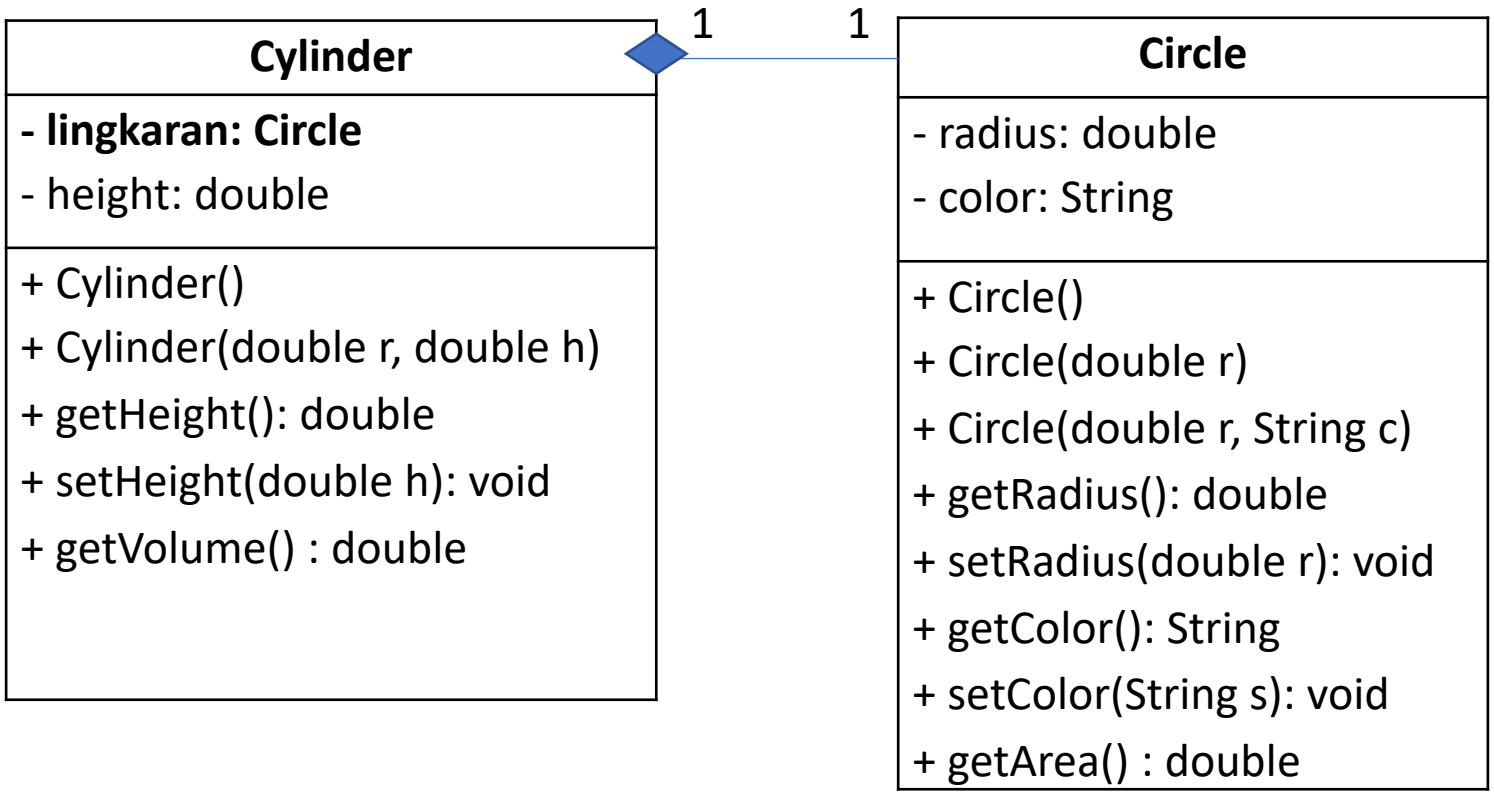
Composition

- Contoh ada class Circle.

Circle
- radius: double - color: String
+ Circle() + Circle(double r) + Circle(double r, String c) + getRadius(): double + setRadius(double r): void + getColor(): String + setColor(String s): void + getArea() : double

- Anggap kita akan buat class Cylinder. Kita dapat re-use class Circle dengan composition.
- Kita sebut: "A cylinder is *composed* of circle with height", or "A cylinder *has* circle with height".
- Composition juga disebut relasi **"*has-a*"**

Contoh Composition



Kode program

```
public class Cylinder
{
   private Circle lingkaran;
   private double height;

   public Cylinder(){
      lingkaran = new Circle();
      height = 10.0;
   }

   public Cylinder(double r, double h){
      lingkaran = new Circle(r);
      height = h;
   }

   public double getHeight(){
      return this.height;
   }

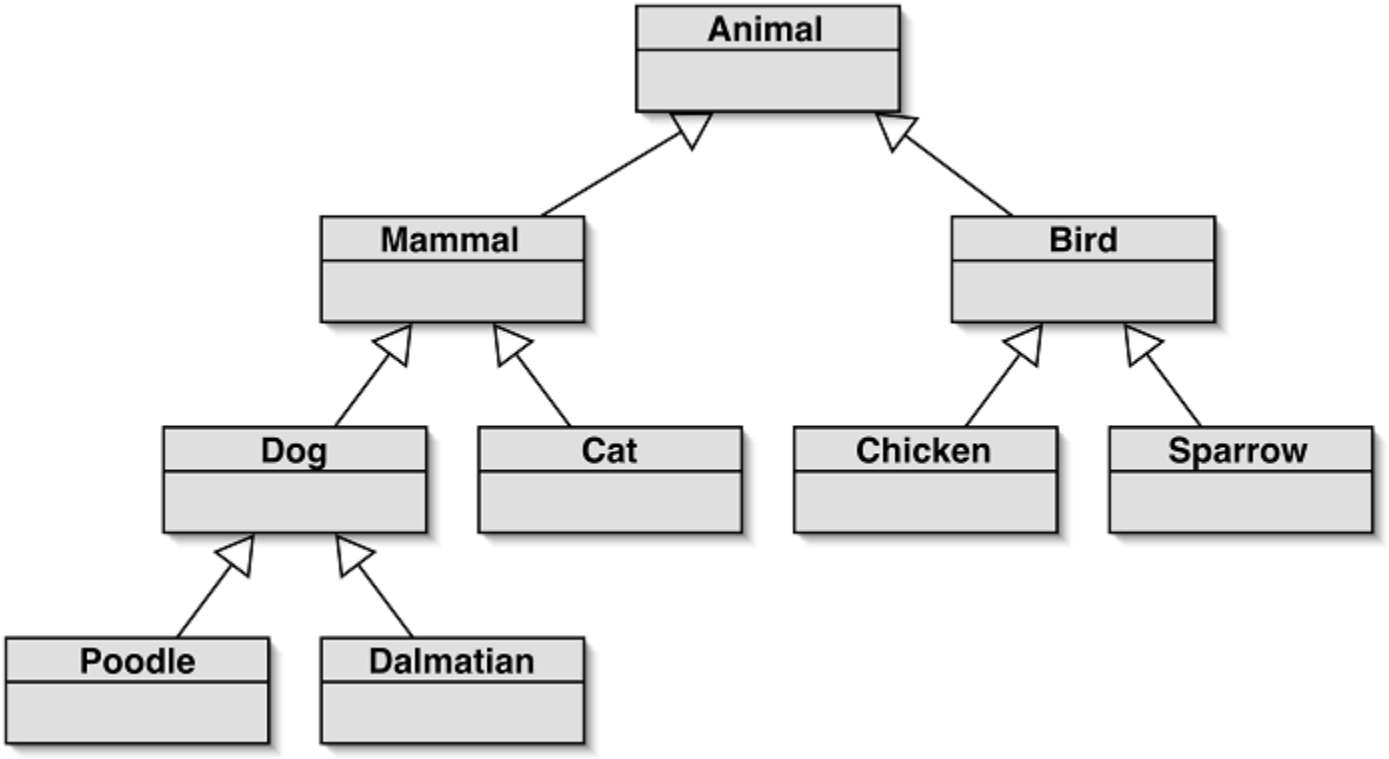
   public void setHeight(double h){
      this.height = h;
   }

   public double getVolume(){
      return lingkaran.getArea() * height;
   }
}
```

Karena volume adalah 22/7*r*r*h
22/7*r*r adalah luas lingkaran sehingga kita dapat panggil method getArea pada class Circle

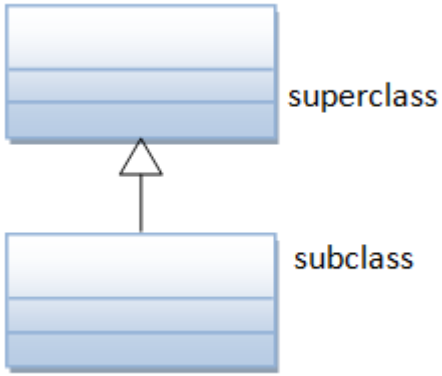
Inheritance

- Atau pewarisan
- Anggap seperti hirarki binatang dalam Biologi.



Inheritance

- Superclass: Class yang berada pada hirarki lebih atas. (Parent)
  - Memiliki variabel dan method yang umum



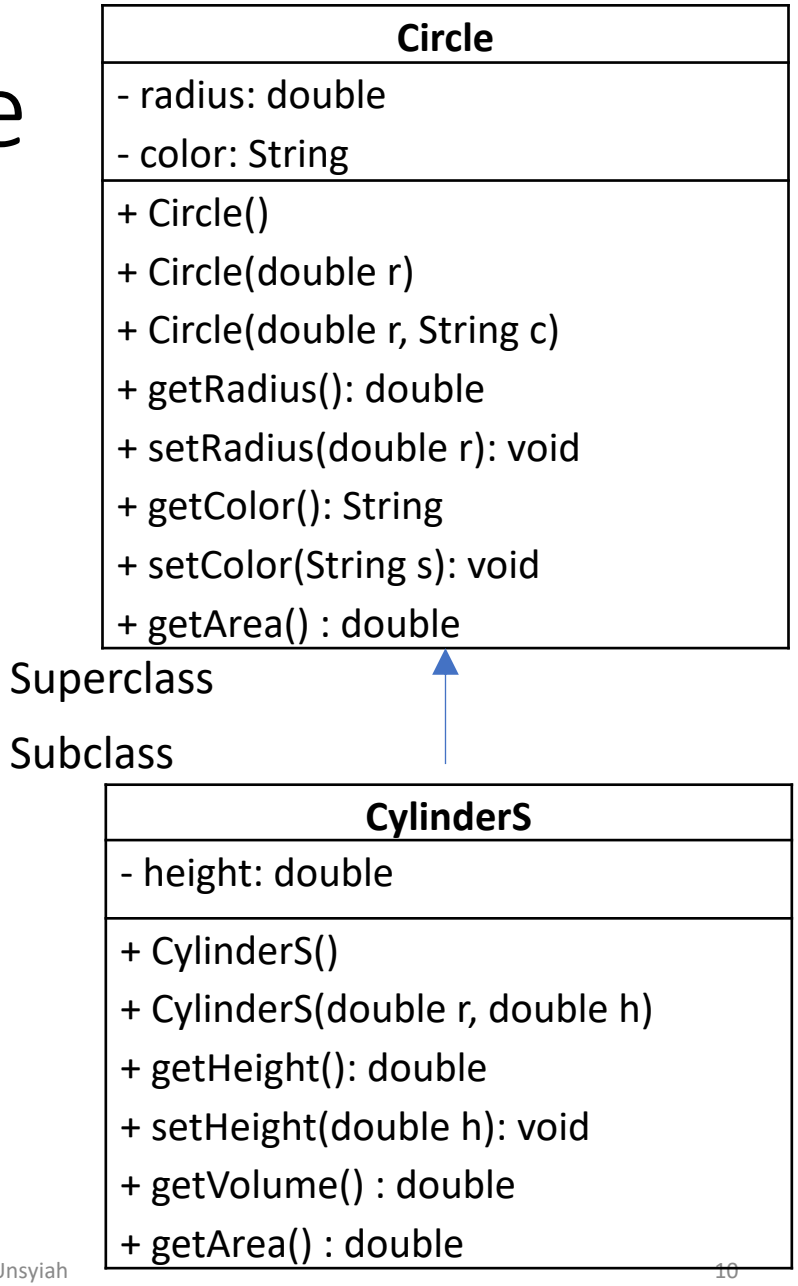
- Subclass: Class yang berada pada hirarki lebih bawah. (Child)
  - Memiliki variabel dan method yang lebih khusus.
  - Subclass mewarisi semua variabel dan method dari superclass-superclassnya.

Inheritance

- Kegunaan inheritance:
  - Menghindari duplikasi code
  - Reuse code
  - Menyederhanakan code
  - Mempermudah maintenance dan perluasan code
- Inheritance juga disebut relasi **“is-a”**
  - Contoh: Poodle is a dog
  - Owl is a bird
  - Bird is a animal

Contoh Inheritance

- Cara subclass mewarisi superclass: dengan keyword "**extends**"
- *A cylinder **is a** circle extended by height*



Keyword "*super*"

- Keyword *super* digunakan untuk merujuk pada variabel dan method **pada superclass**.
- Pada constructor, *super* digunakan untuk memanggil constructor dari superclass.

Constructor pada Inheritance

- Constructor subclass di Java tidak diwarisi dari superclassnya.
- Namun constructor superclass dapat digunakan di dalam subclass dengan menggunakan keyword ***super()***.
- Setiap class di Java memiliki constructornya sendiri.

```
// If no constructor is defined in a class, compiler inserts this no-arg constructor
public ClassName () {
   super();   // call the superclass' no-arg constructor
}
```

Kode Program

```
public class CylinderS extends Circle
{
   private double height;

   public CylinderS(){
      super();
      this.height = 10.0;
   }

   public CylinderS(double r,
   double h){
      super(r);
      this.height = h;
   }

   public double getHeight(){
      return this.height;
   }

   public void setHeight(double h){
      this.height = h;
   }

   public double getVolume(){
      return super.getArea() * height;
   }
}
```

Karena volume adalah 22/7*r*r*h
22/7*r*r adalah luas lingkaran sehingga kita dapat panggil method getArea pada superclass Circle

Method overriding

- Selain dari mewarisi semua variabel dan method dari superclassnya, subclass dapat **mengoverride (menimpa)** variabel dan method.
- Pada class Circle terdapat method getArea() yang berguna untuk mencari nilai luas lingkaran dan mengembalikan nilainya.
- Andaikan kita ingin menghitung luas permukaan Cylinder. Karena sama-sama luas, kita dapat menggunakan method getArea() pada Circle dengan cara meng-overridenya (menimpa dan mengubah isinya).

Kode Program

```
public class CylinderS extends Circle
{
   private double height;

   public CylinderS(){
      super();
      this.height = 10.0;
   }

   public CylinderS(double r,
   double h){
      super(r);
      this.height = h;
   }

   public double getHeight(){
      return this.height;
   }

   public void setHeight(double h){
      this.height = h;
   }

   public double getVolume(){
      return super.getArea() * height;
   }
 public double getArea(){
      return 2*22/7*super.getRadius()*height
+ 2*super.getArea();
   }
}
```



$2*\pi*r*h + 2*\pi*r*r$

@Override

- @Override merupakan *annotation* yang menyuruh compiler untuk mengecek apakah ada nama method yang sama pada superclass.

```
@Override
public double getArea(){
    return 2*22/7*super.getRadius()*height + 2*super.getArea();
}
```

- Optional but nice to have.

@Override vs tanpa @Override

- Jika dengan @Override, akan muncul error kompilasi jika kita tulis getarea() sebab pada Java case sensitive (huruf besar dan huruf kecil beda)

```
    @Override
    public double getarea(){
        return 2*22/7*super.getRadius()*height + 2*super.getArea();
    }
}
```

method does not override or implement a method from a supertype

- Jika tanpa @Override, method getarea() akan dianggap sebagai method baru bukan method yang mengoverride method getArea() pada class Circle

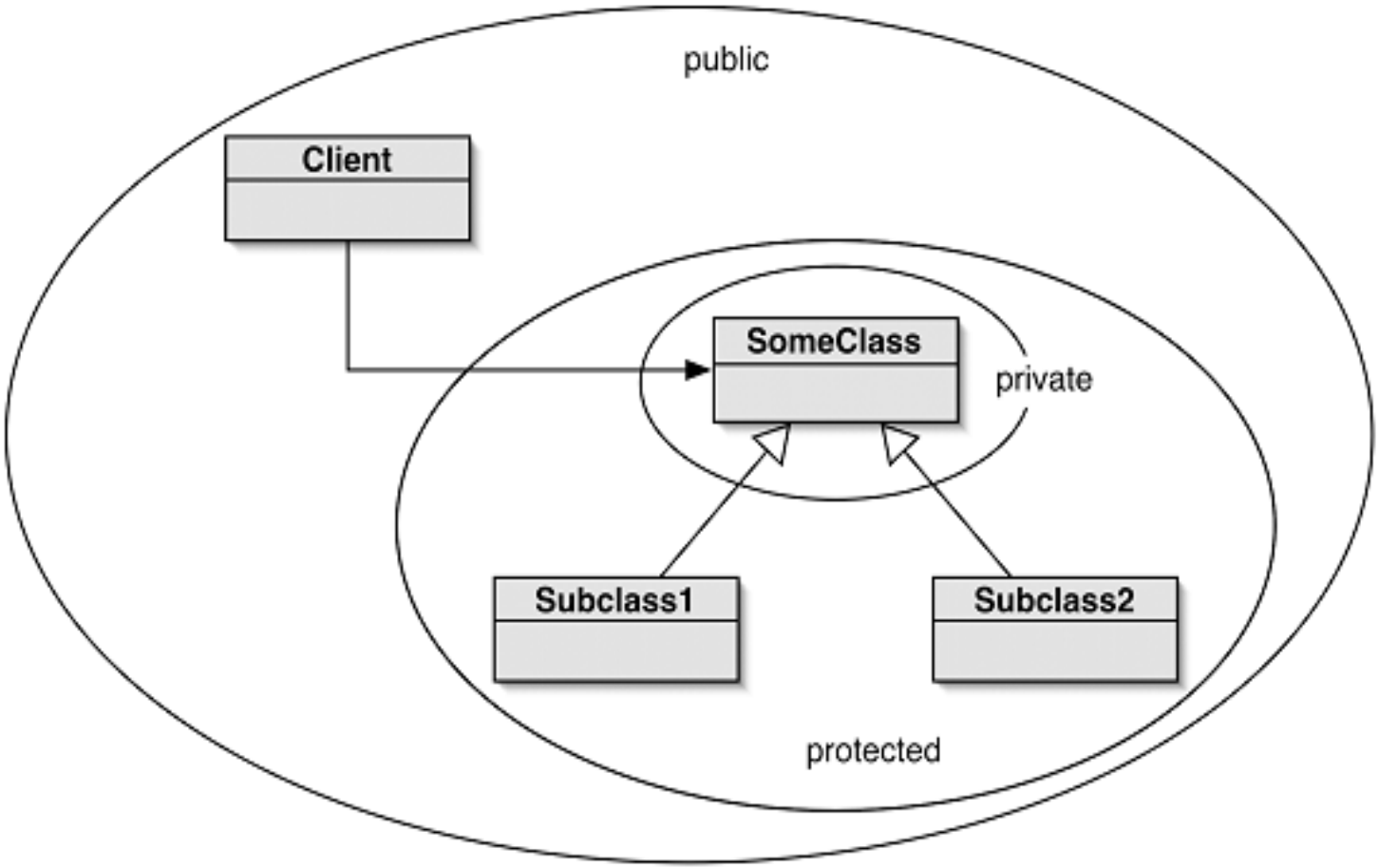
```
    public double getarea(){
        return 2*22/7*super.getRadius()*height + 2*super.getArea();
    }
}
```

Class compiled - no syntax errors

Protected access

- Hak akses private mungkin terlalu terbatas pada superclass.
- Dengan adanya inheritance maka terdapat hak akses baru yaitu *protected access*.
- Hak akses protected hanya berlaku jika ada hubungan inheritance (sub-class dapat merubah data yang dibuat protected pada superclass)
- Namun tetap saja private lebih baik untuk field/atribut.

Access levels



Single Inheritance

- Java **tidak mensupport multiple inheritance** (subclass punya lebih dari satu superclass)
- Pada Java, subclass hanya boleh punya satu superclass langsung di atasnya. Namun superclass boleh punya banyak subclass.

java.lang.Object

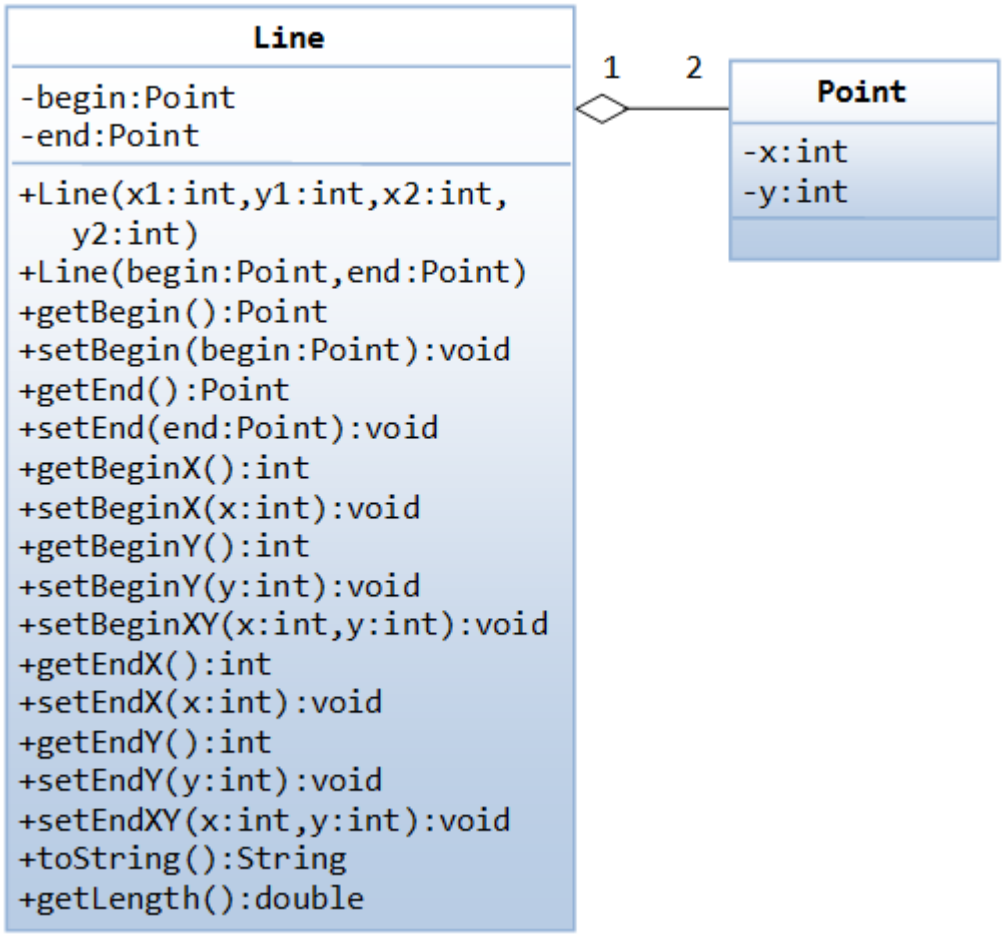
- Setiap class pada Java merupakan subclass dari Object (java.lang.Object).
- Java.lang.Object disebut *common root*.
- Hal ini diperlukan untuk running program dengan JRE, contohnya pada kasus multithreading, garbage collector.

Composition vs Inheritance

- "A cylinder is *composed* of circle with height" vs "A cylinder **is a** circle *extended* by height"?
- Yang mana?
- Pendapat ahli: Use composition if possible, before considering inheritance.

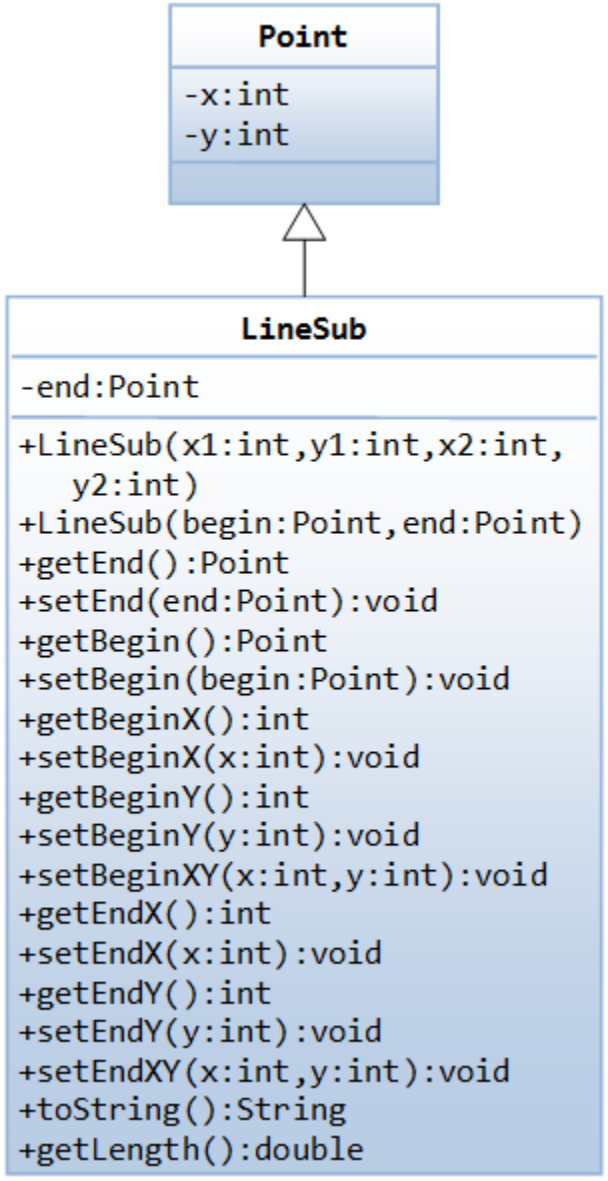
Latihan 1 : Composition

- Buatlah class berdasarkan pada diagram di samping.
- Panah ketupat merupakan notasi untuk Composition
- Angka 2 dan 1 menunjukkan jumlah objek yang dimiliki, artinya: 1 Line dibentuk oleh (Composed by) 2 Point.
- Buatlah sebuah method main() dalam class terpisah untuk menunjukkan contoh hasil yang diinginkan (bebas).
- Buatkan penjelasan kode dalam komentar // atau /* */



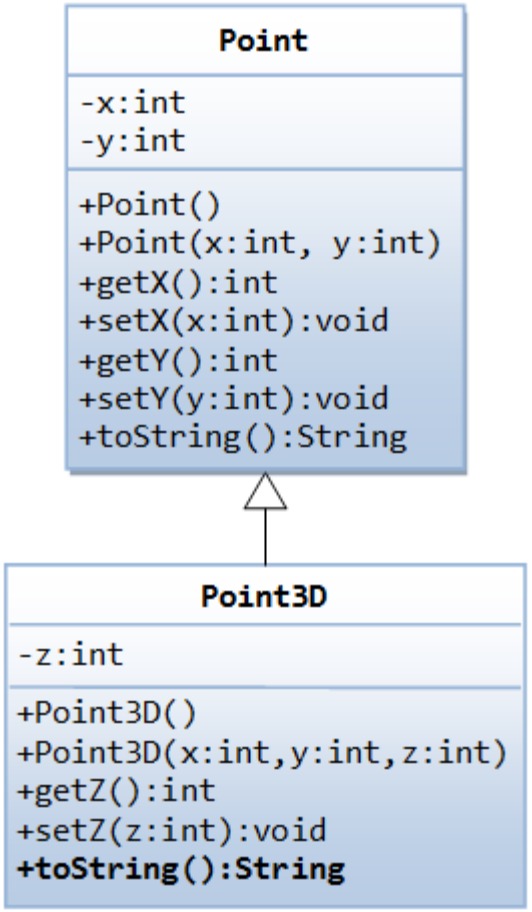
Latihan 2

- Buatlah class berdasarkan pada diagram di samping.
- Panah segitiga merupakan notasi untuk Inheritance. Karena tanda panah mengarah ke Point, maka LineSub merupakan subclass dari Point
  (public class LineSub extends Point)
- Buatlah sebuah method main() dalam class terpisah untuk menunjukkan contoh hasil yang diinginkan (bebas).
- Buatkan penjelasan kode dalam komentar // atau /* */



Latihan 3

- Buatlah class berdasarkan pada diagram di samping.
- Panah segitiga merupakan notasi untuk Inheritance.
- Buatlah sebuah method main() dalam class terpisah untuk menunjukkan contoh hasil yang diinginkan (bebas).
- Buatkan penjelasan kode dalam komentar // atau /* */



I hear, I forget

I write, I remember

I do, I understand..

so..

just do it!

:-)

Important things need to be done at home

- Siapkan komputer anda kemudian:
  - Coba contoh program yang tersedia lalu kerjakan latihan.
- Minggu depan akan ditanya pemahaman anda terhadap program tersebut.